Building UIs, Tools & Multimodal AI

Overview of Today's Topics

Key Learning Objectives

- LLM Frameworks (LangChain, LiteLLM)
- Prompt Caching for cost optimization
- Building UIs with Gradio
- AI Assistants with smart prompting
- Tool Calling (Function Calling)
- Multimodal AI (Images & Audio)

LLM Frameworks Overview

WHY USE FRAMEWORKS?

- Simplify switching between different models
- Consistent interface across providers
- Cost tracking and monitoring

LangChain

1. Feature-rich: agents, memory, chains, tool integration
2. Best for complex workflows and production orchestration
3. Steeper learning curve and larger dependency surface
4. Use cases: multi-tool assistants, long-context apps, production pipelines

LiteLLM

1. Lightweight: simple model switching and minimal abstractions
2. Fast prototyping and lower maintenance overhead
3. Easier onboarding and smaller runtime footprint
4. Use cases: prototypes, research experiments, small services

LiteLLM - Simple Model Switching

KEY BENEFITS:

- One interface for all providers
- Built-in cost tracking
- Easy model comparison

BASIC SYNTAX:

```
from litellm import completion

response = completion(
    model="openai/gpt-4o-mini",
    messages=[{"role": "user", "content": "Hello"}]
)
```

Cost Tracking with LiteLLM

Track every API call: input tokens, output tokens, cost per call

Record tokens and compute cost at call time for accurate billing

Aggregate by user ID, session, and feature flag

Attribute costs to users, sessions, and features for chargeback

Set alerts and budgets to prevent runaway costs

Threshold alerts and automated budget actions

Include sampling and instrumentation to debug high-cost calls

Capture request traces and example prompts for investigation

Emphasize multi-tenant and educational demo use cases

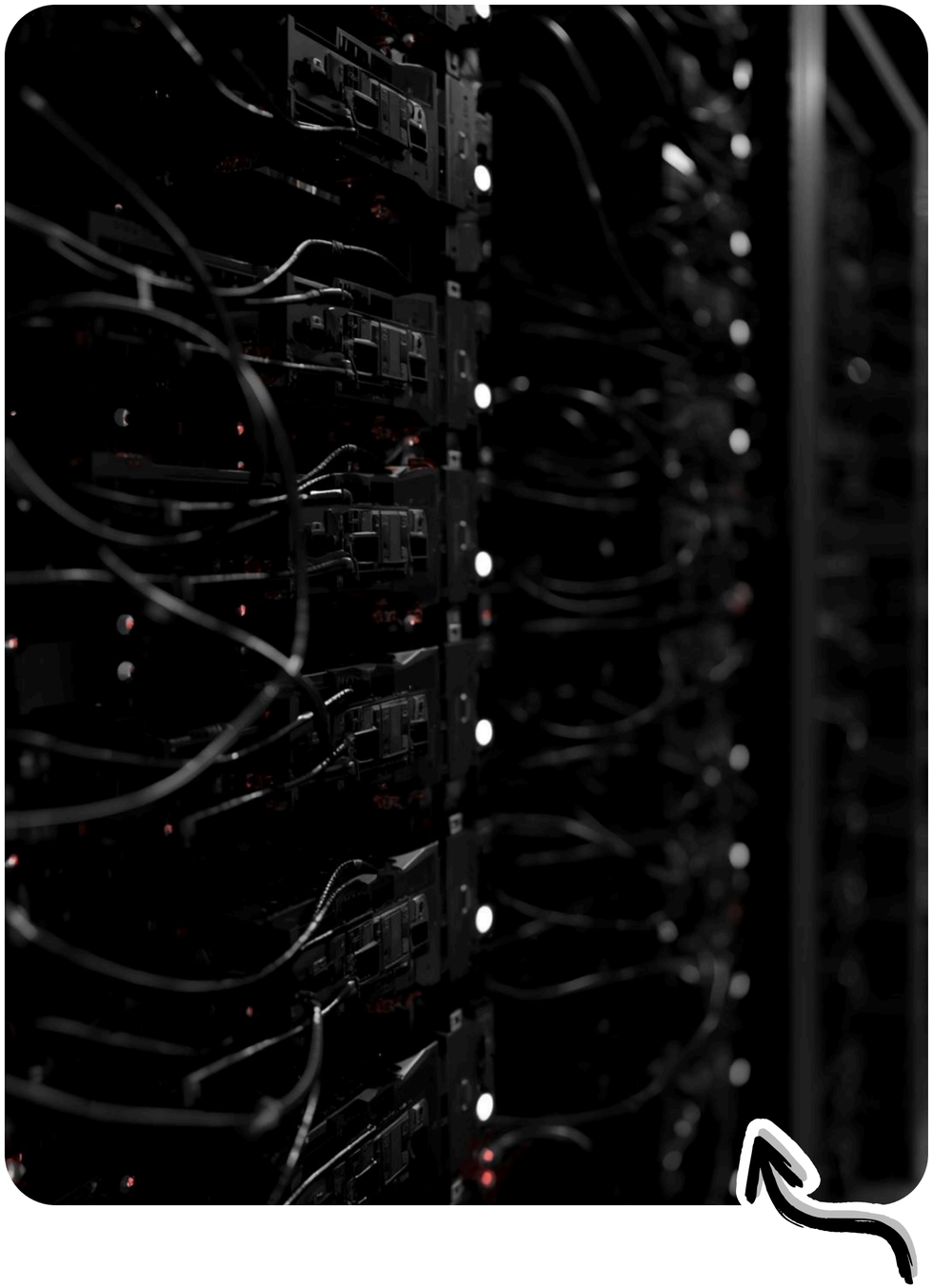
Visibility prevents unexpected charges for tenants and students

Show small KPIs: calls per day, avg tokens per call, cost per call

Surface these in dashboards and reports for stakeholders

Implementation notes: compute cost per call and store time series

Enable rollups by user, session, feature flag, and date



Prompt Caching

What is it?

- Reuse processed prompts across calls
- Pay less for repeated context

Provider differences:

Provider	Type	Benefit
OpenAI	Automatic	~5× savings
Anthropic	Explicit	10× savings (25% prime cost)
Gemini	Both modes	Varies

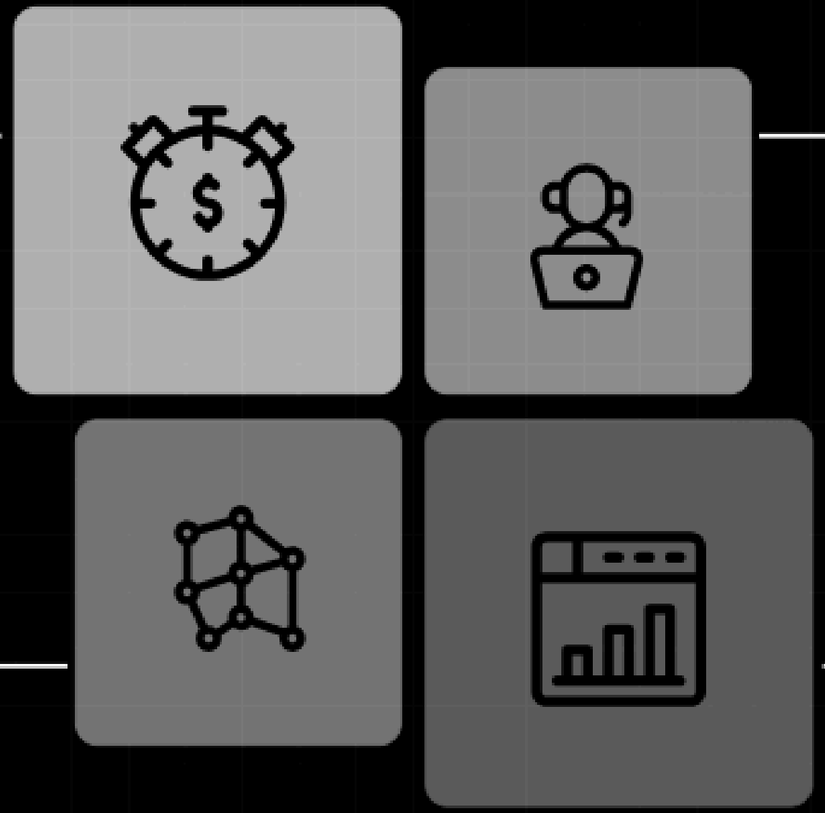
Prompt Caching Best Practice

Cached Prefix

Place large, unchanging context at the start so caching keys match and hits increase.

Variable Suffix

Put ephemeral items like questions and user input at the end to avoid cache fragmentation.



Metadata Separation

Move timestamps, session IDs, and transient data into metadata fields, not the cached prompt.

Practical Example

Wrong: Date then Large Context then Question. Right: Large Context then Question then Date.

LLM Conversations

Two LLMs can talk to each other

- Assign different personas via system prompts
- Build conversation history programmatically
- Educational and entertaining use cases

Example personas:

- One argumentative, one polite
- One optimist, one pessimist

Three-Way Conversations

Challenge:

API only has user/assistant roles

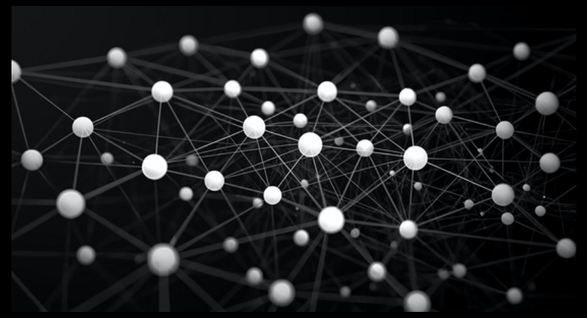
Solution:

Single user prompt with full context

```
System: "You are Alex in a conversation
with Blake and Charlie..."
User: "The conversation so far is: [full
history].
       Respond as Alex."
```

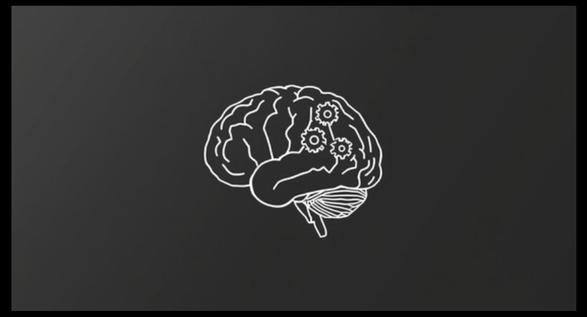
Introduction to Gradio

Key benefit: *Build interfaces without frontend knowledge*



Python library for building UIs

Created by Hugging Face



Perfect for data science demos

Gradio Interface Types

Component	Use case	Quick tip
gr.Interface	Simple input to output demos for a single function	Pick for single-function, fast demos and model examples
gr.ChatInterface	Chat or messaging flows with turn history	Pick for conversational assistants and stateful chat
gr.Blocks	Custom layouts, multiple components, and multimedia	Pick when you need complex layout, multi-component UI, or media

Gradio Features

1

Streaming: Show responses as they arrive

Use for demos to improve perceived responsiveness.

2

Markdown: Render formatted text

Use to present rich outputs like code and lists.

3

Sharing: share=True to create a public URL

Enable for public demos; disable for private classrooms.

4

Authentication: auth=("user","pass")

Protect demos with simple credentials for restricted access.

5

Dark/Light mode: Respects user preference

Test theme behavior across browsers when demoing.

How Gradio Works

Python Description

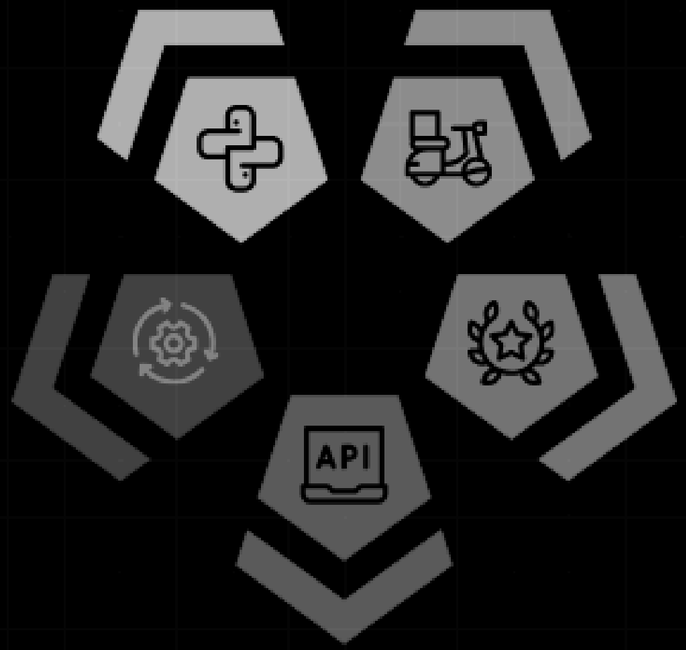
Write UI components and callbacks in Python; source of truth for the app.

Deployment Considerations

Remain in Python while Gradio manages frontend; understand server and route lifecycle when debugging or behind proxies.

API Routes and Callbacks

Gradio creates API routes that invoke your Python callbacks; callbacks execute on the server.



Svelte Frontend

Gradio generates a Svelte-based frontend from the Python description for UI and interactions.

Starlette Server

A Starlette web server runs the app and manages HTTP lifecycle and request handling.

Building AI Assistants

1

Persona (system prompt)

Craft a concise prompt that defines tone, role, and constraints; iterate via user testing for consistency.

2

Memory (conversation history)

Use short-term session history plus long-term external store; prune and summarize for relevance.

3

Domain context (injected docs)

Inject domain docs or retrieval results for factual grounding; keep context concise and source-tagged.

Prompting Techniques

VS

One-shot prompting

- Format: single instruction or style example
- Use for quick style nudges and short tasks
- Pros: concise, low token cost, fast iteration
- Cons: limited pattern control, less consistency

Multi-shot prompting

- Format: multiple Q and A examples
- Use for patterned responses and fine-grained behavior
- Pros: higher consistency, better pattern learning
- Cons: higher token use, needs representative examples
- Recommendation: balance example count with token limits

Dynamic Context Injection for Reliable RAG

- **User query**
  - Receive user message including conditional signals (for example: message containing 'belt')
- **Retrieval**
  - Fetch candidate documents from index or store using the query
- **Relevance scoring**
  - Score and rank documents, select top-k candidates or summaries
- **Context sanitization**
  - Trim for token budget, remove PII, and sanitize text before injection
- **Context labeling**
  - Insert labeled snippets into system prompt so LLM can distinguish grounding from user input
- **Prompt assembly**
  - Combine system message, labeled context, and user query; apply conditional rules (for example: We do not sell belts)
- **LLM response**
  - Model generates answer grounded in injected context to reduce hallucination
- **Example**

```
if "belt" in message.lower():
    system_message += "\nWe don't sell belts."
```

Introduction to Tools

What are tools?

- Give LLMs ability to call external functions
- Database lookups, calculations, bookings
- Foundation of Agentic AI

Lookups: retrieve authoritative data

Database queries or cached records for facts and user state

Actions: perform authenticated external operations

Make bookings or trigger secure API calls on behalf of users

Calculations: deterministic processing off-model

Run precise math, conversions, or business rules outside the LLM

Why tools matter: keep critical logic and authoritative data in code

Reduce trust on model outputs; ensure validation and determinism

Practical examples: validation calls, authenticated actions, deterministic computations

Examples show when to call a tool vs rely on model text

How Tool Calling Really Works

1. **Declare Tools**
   Provide LLM a JSON schema of available tools and inputs
2. **Model Requests Tool**
   LLM responds with a specific tool call like: Please run tool X
3. **Validate Request**
   Check requested tool, parameters, and sandbox constraints before execution
4. **Execute Tool**
   Run the tool in your code with sandboxing and idempotent design
5. **Sanitize Results**
   Clean and redact sensitive data, normalize output formats
6. **Return to Model**
   Pass sanitized results back to LLM for final response generation
7. **Test Edge Cases**
   Simulate retries, failures, and malformed inputs to ensure resilience
8. **Design Idempotency**
   Implement retry-safe tools to reduce side effects during repeated calls
9. **Safety Annotations**
   Log decisions, enforce access controls, and warn on risky operations

Tool Definition (JSON Schema)

```
tool = {
    "type": "function",
    "function": {
        "name": "get_ticket_price",
        "description": "Get price for destination",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string"}
            },
            "required": ["city"]
        }
    }
}
```

Using Tools in API Calls

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    tools=tools  # Pass tool definitions
)

# Check if tool call requested
if response.choices[0].finish_reason == "tool_calls":
    # Handle the tool call
```

Handling Tool Calls

Check for Tool Request

Inspect response.choices0.finish_reason for tool_calls; bail out if not present

Extract Tool Call

Read response.choices0.message.tool_calls0 and sanitize arguments

Execute Function

Run my_function with validated arguments; implement timeouts and retries

Validate Tool Output

Ensure result is well formed, serializable, and safe before returning

Append Result to Messages

messages.append role tool with result then call client.chat.completions.create to continue

Repeat Loop

Loop while finish_reason remains tool_calls; monitor for abort conditions

Common Tool Use Cases

1. **Lookups: Database queries and API calls**
   Secure credentials and use least privilege; log and rate limit requests.
2. **Actions: Book tickets and send emails**
   Confirm user intent and require explicit consent before executing actions.
3. **Calculations: Math operations**
   Validate inputs and handle edge cases to avoid incorrect results.
4. **Code execution: Run Python safely**
   Sandbox execution, limit resources, and review outputs before use.
5. **UI updates: Generate charts and visuals**
   Validate data and preview charts before displaying to users.
6. **Prototyping: Mocks then real services**
   Prototype with mocks first, then connect to real services gradually.

Agentic AI Concepts

LLM runs tools in a loop to achieve goals

Planning

Generate multi-step plan and subgoals from user intent.

Tool Orchestration

Select, call, and combine external tools reliably.

Autonomous Decisions

Make independent choices within **defined** constraints||||.

Memory Persistence

Store context and state across steps for coherence.

Multimodal AI

1

Image Generation — Use For Visual Explanations, Mockups, And Creative Assets

Great For UX And Examples; Watch Content Moderation And Generation Bias

2

Text-To-Speech — Use For Accessibility, Voice Responses, And Notifications

Improves Engagement; Monitor Latency And Voice Quality

3

Vision (Coming) — Use For Image Understanding, Object Detection, And UI Analysis

Enables Visual Inputs; Handle Privacy And Accurate Labeling

4

Transcription (Coming) — Use For Meeting Notes, Search, And Content Indexing

Supports Searchability; Plan For Accuracy And Language Coverage

5

Combined Modalities — Use To Create Richer UX Like Image Plus Caption Plus Spoken Reply

Synchronize Media, Manage Latency, And Enforce Moderation

Building Multimodal Assistants

Chat Interface

Gradio front end for user input, display, and interaction

Tool Calling

Database lookup and function calling for factual retrieval

Image Generation

Create destination photos, then refine for context and style

Captioning

Generate concise captions for images before audio synthesis

Text to Speech

Produce spoken responses; align tone and pacing with caption

UI Render

Orchestrate final render: chat, image, caption, and audio controls

Key Takeaways

1

Frameworks simplify multi-model workflows

Try a toy project to compare frameworks and model switching

2

Gradio builds UIs in minutes not hours

Create a one-screen Gradio demo to showcase features

3

Tools extend LLM capabilities — no magic

Add a single tool (lookup) to your assistant and test flows

4

Prompting techniques improve consistency

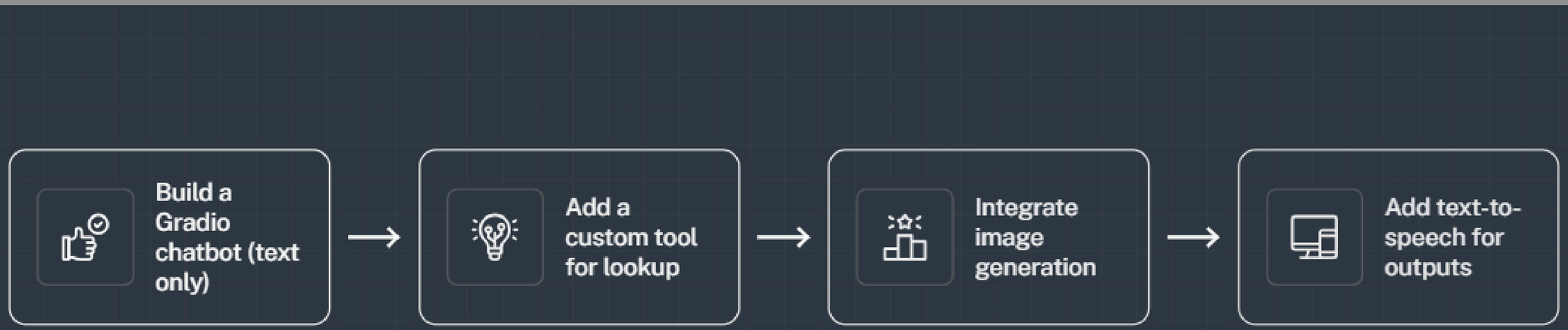
Create prompt templates and automated tests for outputs

5

Multimodal AI creates rich experiences

Prototype one multimodal feature: image or audio input

Next steps


Build a Gradio chatbot (text only)
Add a custom tool for lookup
Integrate image generation
Add text-to-speech for outputs

Thank You